

OmniFormat AI Engine — MVP Implementation Plan

SIH26154 | Team WildCard *Written against the current codebase*

([gen-ai-content-engine/backend/main.py](#), [gen-ai-content-engine/frontend/](#)).

Each phase assumes the previous one is done. Phases 1–4 are the priority — do these before anything else.

Phase 1 — Repo cleanup & ground truth

Goal: the repo should contain exactly what you're presenting. Nothing else.

1.1 Remove or isolate unrelated work

- Move [api.py](#), [start.py](#), [memory-gateway/](#) out of the submission repo (separate repo, or a clearly-named [/experiments](#) folder with a README stating it's unrelated to SIH26154)
- Confirm [gen-ai-content-engine/](#) is the only thing a judge would see if they opened the repo root

1.2 Decide the stack story — Groq vs. Gemini

- Your code uses **Groq** ([openai/gpt-oss-120b / 20b](#)). Your slides say **Gemini**.
- Decision needed now: either (a) update every slide/diagram to say Groq, or (b) swap the backend to actually call Gemini.
- **Recommendation:** keep Groq (it's already working, tested, and has a solid fallback cascade) and update the slides — swapping a working AI integration under time pressure is the higher-risk path.

1.3 Decide the stack story — SQLite/JSON vs. MongoDB

- Your code uses SQLite ([accounts.sqlite3](#)) for accounts and a local JSON file ([chat_history.json](#)) for history. Your slides say MongoDB.
- **Recommendation:** same logic as 1.2 — update slides to reflect SQLite + JSON file storage, unless Phase 7 below (MongoDB migration) is realistically achievable in your remaining time.

1.4 Decide the stack story — custom auth vs. Firebase

- Your custom auth (PBKDF2-HMAC-SHA256 + HMAC-signed OTP + signed session cookies) is genuinely well-built — arguably more impressive than "we used Firebase," since you engineered it yourselves.
- **Recommendation:** keep it, and update slides to say "custom-built secure auth (PBKDF2 + HMAC OTP)" instead of Firebase. This is a strength, not a gap — present it as one.

Output of Phase 1: a written decision (one line per point above) that the whole team agrees on, so slides and code stop contradicting each other.

Phase 2 — True parallel multi-agent generation

Goal: make the "multi-agent, parallel generation" claim in your pitch actually true in code, not just in the diagram.

2.1 Split the single mega-prompt into per-format functions

- Currently: one system prompt with specs for all 10 formats, one Groq call, sequential fallback cascade.
- Target: one function per format (e.g. `generate_advisory()`, `generate_linkedin_post()`, `generate_twitter_thread()`), each with its own focused system prompt containing only that format's rules.
- Keep the existing model cascade logic (120b → 20b, json mode → plain text) — just wrap it into a reusable helper each format function calls.

2.2 Run selected formats concurrently

- In `POST /transform`, replace the single call with `asyncio.gather()` (or `run_in_threadpool` wrapped in `asyncio.gather` since the Groq SDK call is sync) across only the formats the user selected.
- Each format's success/failure should be independent — one failing must not block the others from returning.

2.3 Update the response shape to carry per-format status

- Add a `status` field per format in the response (`"ok"` / `"failed"`), so the frontend can show partial results instead of an all-or-nothing failure.

2.4 Frontend: handle partial results

- `ResultsWorkspace.jsx` — if one tab's format failed, show a small inline error in that tab instead of hiding it, while the others render normally.

Output of Phase 2: generating 5 formats takes roughly the same wall-clock time as generating 1, and you can demo "if I only wanted 2 formats, only 2 agents fire."

Phase 3 — Source-grounding validation (your #1 differentiator)

Goal: every generated output gets checked against the source; unsupported claims are visibly flagged.

3.1 Write the validation function

- New function `validate_grounding(source_text, generated_text) -> list[dict]`
- Simplest MVP approach: one follow-up Groq call per generated output — prompt it with the source and the generated content, ask it to return a JSON list of claims with a supported/unsupported flag and a one-line reason for each unsupported one.

3.2 Wire it into the generation flow

- After each format's content is generated (Phase 2's per-format functions), call `validate_grounding()` on the result before returning it.
- Attach the grounding result to that format's entry in the response payload.

3.3 Frontend: surface grounding flags

- In `ResultsWorkspace.jsx`, if a format has unsupported claims, show a small warning badge/icon on that tab and inline highlight (or a footnote list) of the flagged sentences.

3.4 Demo prep

- Prepare one test document with a subtle factual bait (something the model might over-generalize on) so you can reliably reproduce a flagged claim live if asked to demonstrate this feature.

Output of Phase 3: you can paste a document, generate outputs, and point to a visibly flagged claim on stage — this is the single most important feature to have working before demo day.

Phase 4 — Hash-logging for tamper-evident integrity

Goal: every generated output gets a SHA-256 fingerprint, and you can prove tamper-evidence live.

4.1 Add the hashing utility

```
import hashlib
```

```
def generate_hash(content: str) -> str:  
    return hashlib.sha256(content.encode('utf-8')).hexdigest()
```

4.2 Store the hash alongside each output

- Wherever results are persisted (currently `chat_history.json`), add a `hash` field per format, computed right after generation + grounding check.

4.3 Add a verification endpoint

- `GET /verify/{history_id}/{format_name}` — re-hashes the currently stored content and compares it to the stored hash, returns `{"verified": true/false}`.

4.4 Frontend: show the hash + a "Verify" action

- Small badge under each result tab: `Verified ✓ SHA-256: a3f5e8b2...` (first 8–10 chars is enough visually)
- Optional: a "Re-verify" button that calls 4.3 live — this is a strong, cheap demo moment ("watch, I'll edit this text and re-verify — see it fail now").

Output of Phase 4: your Blockchain & Cybersecurity theme alignment is now backed by real code, not just a slide claim.

Phase 5 — Real export (PDF at minimum)

Goal: stop claiming "native .pptx/.pdf export" unless it's real — get at least one format working.

5.1 Pick the lower-effort target first: PDF

- Add a `GET /export/{history_id}/{format_name}?type=pdf` endpoint

- Use a simple HTML template per format (e.g. Advisory gets a clean header/body layout) rendered to PDF via WeasyPrint (or reportlab if WeasyPrint's system dependencies are painful to install in your environment)

5.2 If time allows: PPTX for the Presentation format

- Only worth doing if the "Presentation" output format already returns structured slide data (title/bullets/notes) rather than free text — check current output shape first
- Use `python-pptx` to map that structure into an actual downloadable deck

5.3 Frontend: wire the export button

- Add a working "Export" button per result tab that calls the new endpoint and triggers a file download

Output of Phase 5: you can genuinely demo downloading a real file, not just copying text to clipboard.

Phase 6 — Wire the missing frontend controls

Goal: use what the backend already supports but the UI doesn't expose yet.

6.1 Add UI controls for existing backend fields

- `language`, `detail_level`, `objective`, `style`, `source_fidelity` are already accepted by `/transform` but have no UI — add dropdowns/toggles for at least `language` and `source_fidelity`, since these are cheap wins that make your platform look more configurable in the demo (and `source_fidelity` ties directly into your anti-hallucination story from Phase 3).

6.2 Confirm defaults are sensible

- Make sure omitted fields fall back to reasonable defaults so nothing breaks for users who don't touch the new controls.

Output of Phase 6: your demo can show tone/audience/language/fidelity controls that judges can see are real levers, not decorative dropdowns.

Phase 7 — Stretch: MongoDB migration (only if Phases 1–6 are done early)

Goal: only attempt this if you have real time left and want the code to match the original slide claims instead of the other way around.

7.1 Stand up MongoDB (local or Atlas free tier)

7.2 Migrate accounts table → **users** collection

7.3 Migrate **chat_history.json** → **history** collection (with the Motor async driver, matching FastAPI's async nature)

7.4 Update slides to confirm MongoDB is now accurate

Do not start this unless Phases 1–6 are solid — a half-migrated database is worse than an honest SQLite/JSON setup that fully works.

Phase 8 — Demo rehearsal & consistency pass

8.1 Full run-through

- One team member plays "judge," asks to see: (a) parallel generation, (b) a flagged hallucination, (c) hash verification, (d) an exported file — confirm all four are demoable without stalling

8.2 Slide/code consistency check

- Walk every technical slide against the actual running app — stack table, architecture diagram, and feature claims must match what's on screen

8.3 Prepare honest answers for what's still not built

- Have one sentence ready for anything left as "roadmap" (e.g. Ollama on-prem deployment, PPTX export if skipped) — see the "What we're explicitly NOT building" section of your team brief as the template for this
-

Priority order if time runs out

If you can't finish everything, do them in this order:

1. Phase 1 (decisions cost nothing but prevent embarrassment)
2. Phase 3 (source-grounding — your single biggest differentiator)
3. Phase 4 (hash-logging — cheap, high payoff for your theme)
4. Phase 2 (true parallel agents — strengthens your core pitch)
5. Phase 5.1 (PDF export only)
6. Phase 6, 5.2, 7 — only if time remains